

MEDIATEK

MT8668 Yocto Property User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	魏新	正式版

目录

版本记录	2
目录	3
图片目录	3
1 Yocto Property	4
1.1 Yocto Property 功能	4
1.1.1 提供 build.prop & local.prop file	4
1.1.2 支持 setprop/getprop cmd	4
1.1.3 支持 persist property	4
1.1.4 Libprop.so 支持的 API	5
1.1.5 支持 Property Watch	5
1.1.6 特定 Property 跨域同步	5
1.2 Yocto Property 框架图	6
1.3 Property Source Code	6
1.4 如何使用 libprop.so	6
1.5 如何添加 Property to local.prop	7
1.6 Property Watch	7
1.7 特定 Property 跨域同步	8
附件一 附加条款	11

图片目录

图 1-1. getprop	4
图 1-2. Property API	5
图 1-3. Property 流程	6
图 1-4. prop.bb	7
图 1-5. watchprop	7
图 1-6. watchprop API	8
图 1-7. Property cross-OS sync 流程 (2OS 和 3OS)	8
图 1-8. setprop from Android to Yocto	9
图 1-9. Yocto UT Test	10

1 Yocto Property

1.1 Yocto Property 功能

1.1.1 提供 build.prop & local.prop file

```
sh-3.2# getprop
[ro.mediatek.version.branch]: [master-spm]
[ro.mediatek.version.release]: [master-spm-2024_05_11_03]
[ro.vendor.mediatek.version.branch]: [master-spm]
[ro.vendor.mediatek.version.release]: [master-spm-2024_05_11_03]
[ro.build.id]: [2024_05_11_03]
[ro.product.device]: [auto8678p1_64_bsp_vm]
[ro.product.model]: [auto8678p1_64_kde_hyp]
[ro.hardware]: [mt8678]
[ro.board.platform]: [mt8678]
[ro.product.name]: [auto8678p1_64_kde_hyp]
[ro.build.flavor]: [auto8678p1_64_kde_hyp]
[ro.build.product]: [auto8678p1_64_kde_hyp]
[ro.build.project]: [auto8678p1_64_kde_hyp]
[ro.build.target]: [auto8678p1_64_kde_hyp]
[ro.mediatek.os]: [yocto]
[ro.build.type]: [userdebug]
[ro.vendor.mbrain.mode]: [mtk]
[vendor.mbrain.ready]: [1]
[ro.kernel.version]: [6.6]
[ro.build.version.release_or_preview_display]: [14]
[sys.boot_completed]: [1]
[ro.build.version.release]: [13]
[ro.vendor.hypervisor]: [1]
```

图 1-1. getprop

1.1.2 支持 setprop/getprop cmd

例如: setprop xxx xxx

```
getprop xxx or getprop | grep xxx    (xxx 指 property key 和 value)
```

1.1.3 支持 persist property

例如: setprop persist.xxx xxx

```
getprop persist.xxx
cat data/property/xxx
```

1.1.4 Libprop.so 支持的 API

```
int property_get(const char *key, char *value, const char *default_value);
int8_t property_get_bool(const char *key, int8_t default_value);
int64_t property_get_int64(const char *key, int64_t default_value);
int32_t property_get_int32(const char *key, int32_t default_value);

/* property_set: returns 0 on success, < 0 on failure
 */
int property_set(const char *key, const char *value);

#ifdef MTK_BINDER_NOSUPPORT
int property_watch_async(const char *key, callback_t_async callback);
/* system=0 for vendor property, system=1 for system property
** boot=0 for no wait android ready, boot=1 for wait android ready
*/
int property_set_sync_android(const char *key, const char *in_value, int system, int boot);
int property_set_only_android(const char* key, const char* value, int system);
int property_set_sync_stou(const char *key, const char *value, int system);
int property_set_only_uos(const char* key, const char* value);
int property_set_sync_utos(const char *key, const char *value);
#endif
int property_list(void (*propfn)(const char *key, const char *value, void *cookie), void *cookie);
```

图 1-2. Property API

1.1.5 支持 Property Watch

见 1.6 章节。

1.1.6 特定 Property 跨域同步

见 1.7 章节。

1.2 Yocto Property 框架图

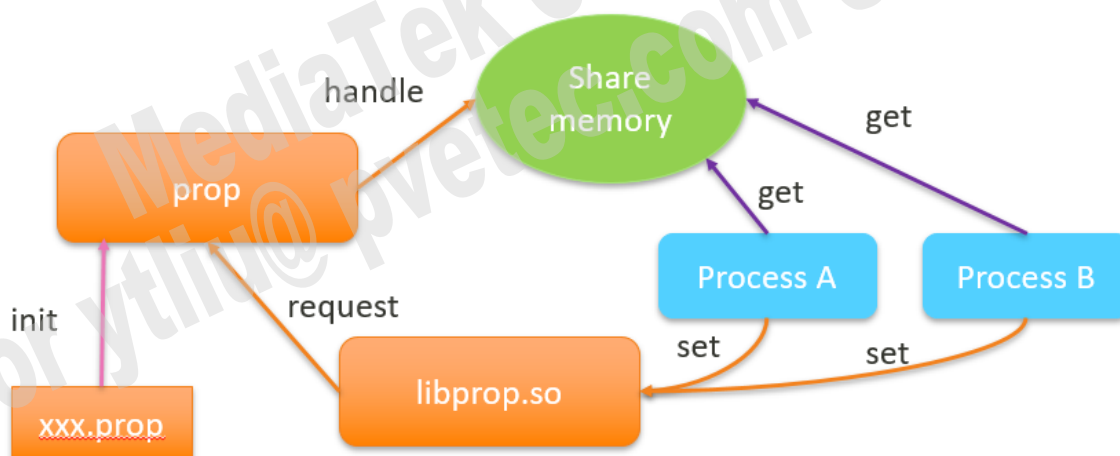


图1-3. Property 流程

Yocto property 包括一个 prop daemon service 和一个 libprop.so。Module link libprop.so 就可以调用其中的 API，当某个 process 调用 property set 会与 prop 通信，prop 将 property 写入共享内存，当某个 process 调用 property get，直接从共享内存中读取。

1.3 Property Source Code

Yocto:

- Source path: `src/apps/atom-base/progs/property/`
- bb path:
 - `meta/meta-mediatek/recipes-devtool/prop/prop.bb`
 - `meta/meta-mediatek/recipes-devtool/prop/propsync.bb`
 - `meta/meta-mediatek/recipes-devtool/prop/propsyncs.bb`
 - `meta/meta-mediatek/recipes-devtool/prop/propsyncu.bb`
 - `meta/meta-mediatek/recipes-devtool/libprop/libprop.bb`

Android:

- Source path: `vendor/mediatek/proprietary/external/propsync/`

1.4 如何使用 libprop.so

如果一个 module 想调用 property API，需要 link libprop.so，方法如下：

1. bb 中修改

```
DEPENDS = "libprop"
```

2. makefile 中加:

```
LDFLAGS: -lprop
```

3. source code 增加 properties.h

```
#include <prop/properties.h>
```

4. xxx.service 依赖 prop.service (特别是开机比 prop 起来早, 需要等 prop ready 后才能使用)

```
After=prop.service
Requires=prop.service
```

1.5 如何添加 Property to local.prop

在 prop.bb 中将 build time 所需要产生的 property 放在 local.prop。

```
do_compile:append() {
    if [ -f ${VISION_SRC} ]; then
        buildtime="$(cat ${VISION_SRC} | cut -d':' -f 2)"
        buildbranch="$(cat ${VISION_SRC} | cut -d'/' -f 1)"
        buildproject="$(cat ${VISION_SRC} | cut -d'/' -f 2)"
        buildtarget="$(cat ${VISION_SRC} | cut -d'/' -f 2)"
        echo '[ro.mediatek.version.branch]: ['$buildbranch']' >> ${S}/local.prop
        echo '[ro.build.id]: ['$buildtime']' >> ${S}/local.prop
        echo '[ro.product.device]: ['$MTK_PROJECT']' >> ${S}/local.prop
        echo '[ro.hardware]: ['$TARGET_PLATFORM']' >> ${S}/local.prop
        echo '[ro.build.type]: ['$MTK_PROJECT']' >> ${S}/local.prop
        echo '[ro.build.project]: ['$buildproject']' >> ${S}/local.prop
        echo '[ro.build.target]: ['$buildtarget']' >> ${S}/local.prop
    fi
}
```

图 1-4. prop.bb

1.6 Property Watch

1. property_watch_async() 函数开发是用来监测某个 key, 当这个 key 被设置, 会通知调用 property_watch_async 的 process, 执行 callback 函数。

UT test 方法: 在 Yocto 端, 执行 watchprop -w xxx(key) xxx(value)

```
sh-3.2# watchprop -w km.test test
watchprop callback key=km.test, value=testsh-3.2#
```

图 1-5. watchprop

2. 如果 module 想监测某个 property 做事, 可以直接调用 property_watch_async 函数, 实现自己的 callback 函数

注意: 调用 property_watch_async 是使用的异步 callback。

```
int property_watch_async(const char *key, callback_t_async callback);
typedef void (*callback_t_async)(const char *key, const char *value);
```

图 1-6. watchprop API

1.7 特定 Property 跨域同步

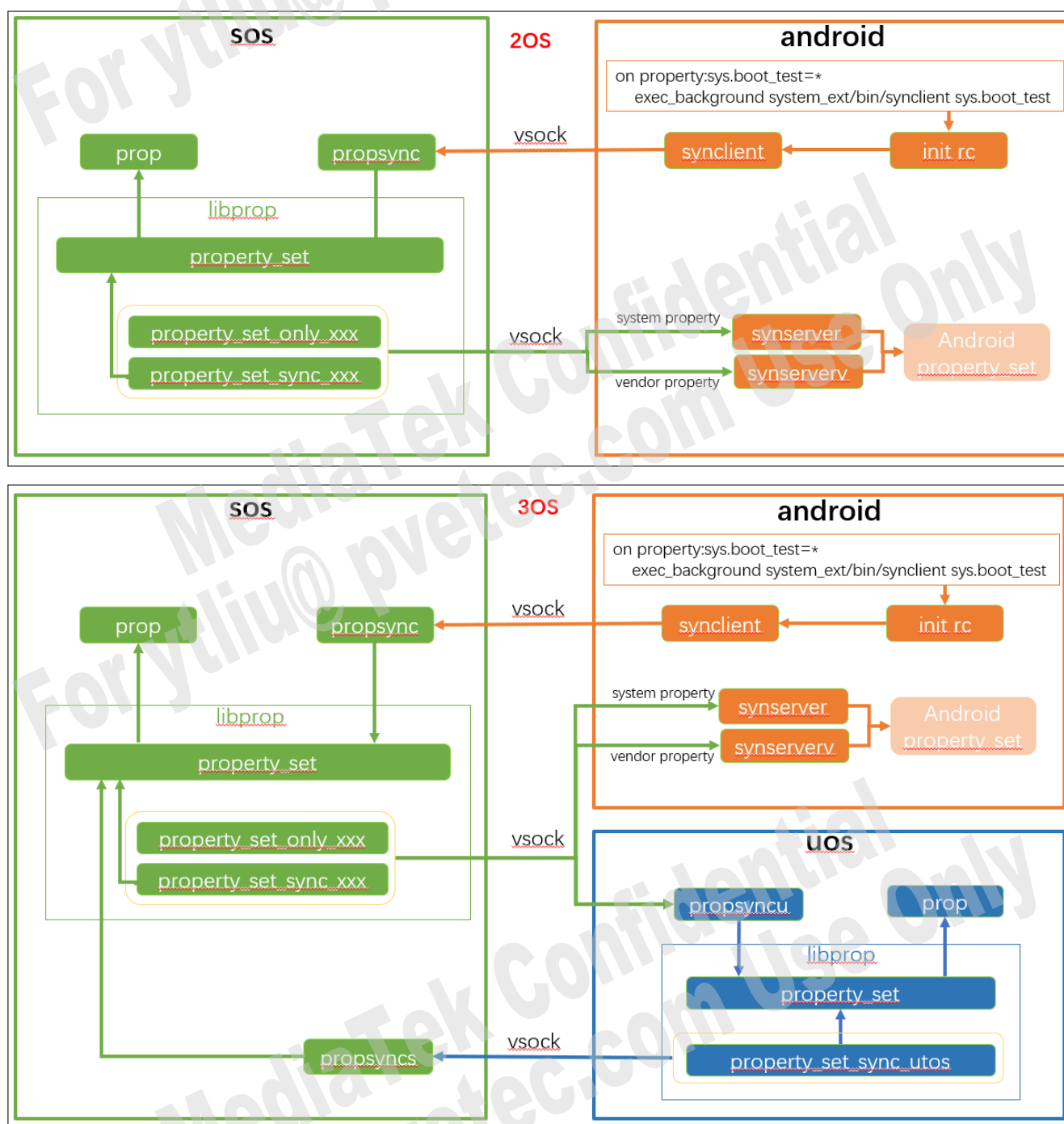


图 1-7. Property cross-OS sync 流程 (2OS 和 3OS)

1. Android sync to Yocto (在 Android rc 中设定要监测的 property, 当这个 property 被设置时会 trigger process synclient 跑起来, 通过 vsocket 与 Yocto propsyncserver 通信, 然后 Yocto propsyncserver 会设置 property)

UT test:

- 1) 在 init.project.rc 中监测 sys.boot_test

```
71 on property:sys.boot_test=*
72     exec system_ext/bin/synclient sys.boot_test
```

- 2) 在 Android 端: adb shell setprop sys.boot_test done

- 3) 在 Yocto 端检查: adb shell getprop sys.boot_test



```
km2
auto8678p1_64_bsp_vm:/ # setprop sys.boot_test done
auto8678p1_64_bsp_vm:/ #
sh-3.2# getprop sys.boot_test
done
sh-3.2#
```

图 1-8. setprop from Android to Yocto

2. Yocto sync to Android (直接调用 property_set_sync_android 或者 property_set_only_android 函数, 其函数会通过 vsocket 与 Android synserver 或 synserverv 通信, 然后 synserver 或 synserverv 会设置这个 property)

UT test:

- 1) 在 Yocto 端

```
adb shell watchprop -sa xxx(key) xxx(value) 0 0 (vendor property)
```

```
adb shell watchprop -sa xxx(key) xxx(value) 1 0 (system property)
```

- 2) 在 Android 端检查: adb shell getprop xxx

3. Yocto sos sync to uos (直接调用 property_set_only_uos 函数, 其函数会通过 vsocket 与 uos propsyncserver_uos 通信, 然后 uos propsyncserver_uos 会调用 property_set 函数 set property)

UT Test:

- 1) 在 Yocto sos 端

```
adb shell watchprop -u xxx(key) xxx(value)
```

- 2) 在 Yocto uos 端检查: adb shell getprop xxx

4. Yocto uos sync to sos (直接调用 property_set_sync_utos 函数, 其函数会通过 vsocket 与 sos propsyncserver_sos 通信, 然后 sos propsyncserver_uos 会调用 property_set 函数 set property)

UT Test:

- 1) 在 Yocto uos 端
adb shell watchprop -us xxx(key) xxx(value)
- 2) 在 Yocto sos 端检查: adb shell getprop xxx

5. Yocto UT test 相关命令

在 Yocto shell 敲 watchprop -h 可以查看相关测试说明

```
printf("=====The following fixed-format commands are supported=====\\n");
printf("test API: | cmd:\\n");
printf("property_watch_async&property_set: | watchprop -w <key> <value>\\n");
printf("property_set_sync_android: | watchprop -sa <key> <value> <system> <boot>\\n");
printf("property_set_only_android: | watchprop -a <key> <value> <system>\\n");
printf("property_set_sync_stou: | watchprop -sau <key> <value> <system>\\n");
printf("property_set_only_uos: | watchprop -u <key> <value>\\n");
printf("property_set_sync_utos: | watchprop -us <key> <value>\\n");
```

图 1-9. Yocto UT Test

注意: 测试前请先关掉 selinux (adb shell setenforce 0)!

6. 跨域相关 API

API	使用的 OS	Set Property to		
		sos	uos	android
int property_set_only_android(const char* key, const char* value, int system);	sos			Y
int property_set_only_uos(const char* key, const char* value);	sos		Y	
int property_set_sync_android(const char *key, const char *in_value, int system, int boot);	sos	Y		Y
int property_set_sync_stou(const char *key, const char *value, int system);	sos	Y	Y	Y
int property_set_sync_utos(const char *key, const char *value);	uos	Y	Y	

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。